

LUDO - SDP

TEAM PANDA

NOVEMBER 1, 2026

PANDA
Hila Kasa, Shira Marder, Stav Elkriyef, Lana Admuz

Contents

1. Overview 2

1.1 Project Summary 2

1.1.1 Purpose, scope and objectives - The project develops an educational Ludo game for 2-4 players, combining traditional gameplay with Software Engineering and Software Quality Assurance questions. It includes three difficulty levels, game history, and question management in accordance with the SRS. 2

1.1.2 Assumptions and constraints 2

1.1.3 Project deliverables – 2

1.1.4 Schedule and budget summary – 2

2. References – 2

3. Definitions 2

5.2.2 Schedule allocation – 6

5.3 Control plan 7

5.3.4 Quality Control Plan - Iteration 1 - Definition of Done – 7

5.4 Risk Management Plan 8

6.1 Process model – 10

6.2 Methods, tools and techniques – 10

7. Supporting process plan 11

7.3 Documentation Plan – 11

1. Overview

1.1 Project Summary

1.1.1 Purpose, scope and objectives - The project develops an educational Ludo game for 2-4 players, combining traditional gameplay with Software Engineering and Software Quality Assurance questions. It includes three difficulty levels, game history, and question management in accordance with the SRS.

1.1.2 Assumptions and constraints

Assumptions

- All four team members will have access to suitable computers and the planned development and collaboration tools throughout the project.
- Team members will be available for regular coordination and completion of assigned tasks across the three iterations.
- Computers running the application will support the file access required for storing and retrieving game history and question bank data.
- Development tasks may be reassigned within an iteration based on workload and team availability, with changes coordinated and documented.

Constraints: The application must be implemented in Java using MVC architecture and appropriate design patterns, with source code managed through GitHub. Development must follow Scrum across three iterations. Completed game history must use the course-specified CSV schema, and all deliverables must meet the course deadlines.

1.1.3 Project deliverables –

The deliverables include the SRS and SDP documents, an interactive prototype, three runnable software increments, a complete Java application, and source code managed through GitHub. Deliverables will be submitted according to course guidelines.

1.1.4 Schedule and budget summary –

The seven-week project (November 2-December 21, 2026) comprises three iterations, each delivering a runnable software increment. The complete application is scheduled for completion by the end of Iteration 3. The detailed schedule is presented in Section 5.2.2.
The four-member student team will use Java, GitHub, and existing computing resources. Development time is the primary resource, with no additional monetary expenses planned.

2. References – Ludo Project Requirements - University of Haifa, Software Engineering and Software Quality Assurance in Information Systems, 2026.
Software Requirements Specification (SRS) - Team Panda, October 2026.
IEEE Std 1058-1998 - IEEE Standard for Software Project Management Plans, IEEE, 1998.
Exercise 3 - SDP and Prototype - University of Haifa, Software Engineering and Software Quality Assurance in Information Systems, October 2026.

3. Definitions

SDP: A document detailing project activities, schedules, resources, and management.

SRS: A document defining the system's functional and non-functional requirements.
Agile: An iterative development approach emphasizing incremental delivery and adaptability.
Scrum: An Agile framework managing development via iterations, roles, and collaboration.
Iteration: A development cycle delivering a working software increment from defined tasks.
User Story: A brief description of a system feature from the user's perspective;
MVC: An architectural pattern separating data, user interface, and control logic.
Definition of Done: A set of criteria that must be met before a task is considered complete.
V&V: Processes ensuring the software meets specified requirements and actual user needs.

5. Managerial process plan

5.2 Work plan

5.2.1 Work Activities

Iteration 1	**Iteration goal:** Develop a working foundation of the Ludo game, including game setup, an Easy board, dice rolling, basic piece movement, and turn management.
US-01	As a player, I want to configure a new game by selecting the number of players, their names, colors, and an available difficulty level, so that I can start a game with my preferred settings.
US-02	As a player, I want to view an Easy game board with all players' pieces in their initial positions, so that I can see the initial game state.
US-03	As a player, I want to roll a standard die, so that I can determine how many steps I can move.
US-04	As a player, I want to enter my pieces onto the board and move them forward according to the dice result, so that I can progress along the game path.
US-05	As a player, I want the game to manage turns and additional rolls according to the basic dice rules, so that each player can take their turn correctly.
Iteration 2	**Iteration goal:** Extend the Ludo game by implementing advanced game rules, Medium difficulty, educational questions, surprise events, game completion, and unit testing.
US-06	As a player, I want the game to enforce capturing, protected squares, and Double Block rules, so that all piece interactions follow the Ludo rules.
US-07	As a player, I want to move my pieces into their home paths and complete the game according to the finishing rules, so that the game can determine the final player rankings.
US-08	As a player, I want to play in Medium difficulty with an expanded board and Boost dice, so that I can experience more challenging gameplay.
US-09	As a player, I want to answer difficulty-based questions when I land on Question squares, so my answers affect my progress in the game.
US-10	As a player, I want Surprise squares to trigger unexpected rewards and challenges.

US-11	As a player, I want my completed game results to be saved, so that they can be accessed in future sessions.
Iteration 3	**Iteration goal:** Complete the Ludo application by implementing Hard difficulty, secure question bank administration, game history viewing, integrating design patterns, and performing final system testing.
US-12	As a player, I want to play in Hard difficulty with the fully expanded board and UltraBoost dice, so that I can experience the highest level of gameplay complexity.
US-13	As an administrator, I want to securely access, view, add, edit, and delete questions, so that I can manage the educational question bank.
US-14	As a player, I want to view the history of completed games, including player rankings, difficulty levels, and game durations, so that I can review previous game results.

Requirement/ User Story// SRS ID		**Task**	**Executor**
US-01- Game Setup	FR 1.1, FR 1.2	Create the game setup screen with player count, name, and color inputs.	Shira
		Implement player count validation (2-4) and unique color selection.	Shira
	FR-1.3	Display all difficulty levels, enable Easy only, and prevent difficulty changes once the game starts.	Shira
	FR 1.1- FR 1.3	Implement player and game configuration models.	Shira
		Connect the Start Game action to Easy game initialization.	Shira
		Test game setup, input validation, and difficulty selection.	Shira
		Perform acceptance testing of the game setup flow.	Prod Owner
US-02- Easy Game Board	FR 1.4	Implement the Easy board model and its 52-square common path.	Stav
	FR 1.4, FR 1.5	Place four Starting squares and four color-specific Arrow squares at predefined positions.	Stav
	FR 1.4, FR 1.6	Randomly place four Safe squares at distinct eligible positions without overlapping fixed squares.	Stav
	FR 1.7	Create player castles and six-square home paths.	Stav
	FR 1.8	Initialize four pieces per player in their corresponding castles.	Stav

	FR 1.4 - FR 1.8	Display the Easy board, square markings, castles, home paths, and initial pieces.	Stav
		Integrate board initialization with the game configuration from US-01.	Stav
		Test board structure, square placement, and initial piece positions.	Stav
	FR 1.4, FR 1.8	Perform acceptance testing of the initial Easy board. Integrate board initialization with the game configuration from US-01.	Prod Owner
US-03- Dice Rolling	FR 8.1, FR 8.2	Implement the Steps Die for Easy mode, generating random integers from 1 to 6 with equal probability.	Shira
	FR 8.2	Create the Roll Dice button and display the dice result.	Shira
		Connect the Roll Dice button to the dice generation logic.	Shira
	FR 8.1, FR 8.2	Store the dice result in the game state and make it available to the movement and turn management components.	Shira
		Test dice generation, result display, and integration with the game state.	Shira
		Perform acceptance testing of the dice rolling functionality.	Pro Owner
US-04- Piece Movement	FR 2.1, FR 8.7	Implement piece selection and enforce movement of only one piece per valid roll	Hila
	FR 2.1	Implement castle exit when the Steps Die result is six, placing the selected piece on its starting square	Hila
	FR 8.4	Calculate movement steps in Easy mode based on the Steps Die result	Hila
	FR 2.3	Implement clockwise piece movement along the Easy board path	Hila
		Update piece positions in the game state and board display after movement	Hila
	FR 2.1, FR 2.3	Validate basic piece movement and castle exit conditions	Hila
	FR 2.1, FR 2.3, FR 8.4	Connect piece movement logic to the board and dice components	Hila
	FR 2.1, FR 2.3, FR 8.4, FR 8.7	Test castle exit, movement direction, step calculation, and single-piece movement	Hila

		Perform acceptance testing of basic piece movement	Prod`Owner
US-05- Turn Management	FR 8.5, FR 8.9	Implement active player tracking and turn rotation for 2-4 players.	Lana
	FR 8.5, FR 8.8	Control dice rolling permissions based on the active player and turn state.	Lana
	FR 8.5	Implement an additional roll when the Steps Die result is six.	Lana
	FR 8.6	Track consecutive sixes, cancel the third consecutive six and its movement, and pass the turn.	Lana
	FR 8.8	Check for available legal moves and prevent voluntary turn skipping when a legal move exists.	Lana
	FR 8.9	Handle turns with no legal moves, allowing another roll after six or passing the turn otherwise.	Lana
	FR 8.5, FR 8.8, FR 8.9	Integrate turn management with dice rolling and piece movement, and display the active player.	Lana
	FR 8.5, FR 8.6, FR 8.8, FR 8.9	Test turn rotation, additional rolls, consecutive sixes, and legal movement handling.	Lana
	FR 8.5, FR 8.6, FR 8.8, FR 8.9	Perform acceptance testing of turn management.	Prod Owner

Some SRS requirements will be partially implemented in Iteration 1, with their remaining functionality planned for subsequent iterations. The task breakdown reflects only the functionality included in the first iteration.

5.2.2 Schedule allocation – The project is planned across three iterations, following the course schedule. Each iteration includes development, integration, testing, and delivery of a runnable increment.

Iteration	**Planned Period**	**Main Activities**	**Milestone**
Iteration 1	Nov 2-16, 2026 (2 weeks)	Game setup, Easy board, dice rolling, basic movement, and turn management (US-01-US-05).	Runnable basic Easy gameplay.

Iteration 2	Nov 16- Dec 7, 2026 (3 weeks)	Advanced game rules, game completion, Medium difficulty, questions, surprise events, results storage, and unit testing (US-06-US-11).	Runnable extended game with unit tests.
Iteration 3	Dec 7- Dec 21, 2026 (2 weeks)	Hard difficulty, question bank administration, game history, design patterns, and final system testing (US-12-US-14).	Complete, tested, runnable application.

Dependencies and Parallel Work - In Iteration 1, game setup, board development, and dice rolling can proceed in parallel, while movement and turn management depend on these components. In Iteration 2, advanced movement rules support Medium gameplay, questions, and surprise events, while results storage depends on game completion. Iteration 3 features can be developed concurrently before final integration. Progress will be reviewed weekly against milestones using completed functionality and test results. Dates are based on the course syllabus and may be adjusted to official deadlines.

5.3 Control plan

5.3.1 Requirements control plan

SRS changes require Product Owner and Scrum Master review and approval by all four members. The Product Owner assesses scope, quality, schedule, workload, and risks; the Scrum Master assesses Java/MVC implications. After approval, the Product Owner updates the SRS, assigns a unique tracking ID, and adds related backlog tasks.

5.3.4 Quality Control Plan - Iteration 1 - Definition of Done –

A task in Section 5.2.1 is Done when:

1. It meets its linked SRS requirements within the Iteration 1 scope.
2. Normal and edge case tests pass; SRS references, expected, and actual results recorded
3. The applicable peer review below is completed.
4. Requirement-blocking defects are corrected and retested. Deferred minor findings have Product Owner approval, an owner, and a follow-up date.
5. Updated code and relevant documentation are shared with the team.

A user story is Done when all its tasks meet these criteria and the Product Owner records acceptance of its integrated Iteration 1 functionality (US-01-US-05) for 2-4 players.

Control Intensity -Development tasks are grouped below by complexity, requirement importance, and risk. Testing and acceptance tasks follow the criteria above and responsibilities in Section 7.2.

US-01 - Game setup (FR 1.1-FR 1.3): One peer reviews input validation, configuration models, and initialization using a checklist covering valid inputs, unique colors, difficulty restrictions, and settings transfer. The simple logic warrants a focused review, while interface checks address RSK-02.

US-02 - Easy board (FR 1.4-FR 1.8): The author and two peers conduct a walkthrough of example board configurations, checking placement constraints, initial pieces, and model-display consistency. Interacting constraints and movement dependencies justify greater review effort (RSK-01, RSK-02).

US-03 - Dice rolling (FR 8.1-FR 8.2): One peer checks random generation, event handling, and result storage and transfer for valid values, equal probabilities, and consistent results. Although small, this component affects movement and turns (RSK-01, RSK-02).

US-04-US-05 - Movement and turns (FR 2.1, FR 2.3, FR 8.4-FR 8.9): The author and two peers read the code and note questions beforehand, then conduct a walkthrough tracing normal and edge cases against a checklist of rule conditions, path boundaries, state updates, and interfaces. These interacting rules receive the greatest review effort because failures can block gameplay (RSK-01, RSK-02).

Limited-depth review: The Roll Dice button and result display (US-03, FR 8.2) receive only a brief peer execution check. Residual presentation risk is acceptable because defects are visible and easily corrected; dice logic and event handling remain covered above.

5.4 Risk Management Plan

RSK-01 Incorrect Movement and Turn Rules

Description: Incorrect implementation of interacting movement and turn rules may permit illegal actions or block gameplay, delaying delivery through rework (US-04-US-05).

Probability- Medium: Rules share conditions and state transitions.

Impact- High: Failures disrupt core gameplay and dependent features.

Strategy and action-Mitigation: Use rule-checklist walkthroughs and independent normal/edge case tests, including consecutive sixes and no legal moves.

Monitoring and response: Review defects weekly; recurring failures trigger focused review, correction, and regression testing. **Owner:** Stav Elkriyef.

RSK-02 Integration Inconsistency

Description: Incompatible interfaces or inconsistent shared state may prevent separately developed components from forming a runnable increment, delaying delivery (US-01-US-05).

Probability - Medium: Multiple Devs implement components exchanging game data.

Impact - High: Integration failures can prevent playable gameplay.

Strategy and action - Mitigation: Define interfaces and state ownership; incrementally test the setup-roll-move-turn flow for 2-4 players and model-display consistency.

Monitoring and response: Check integration weekly and after interface changes; prioritize blocking defects and repeat integration/regression tests after correction. **Owner:** Stav Elkriyef.

RSK-03 Schedule delays due to limited team availability

Description: Overlapping academic commitments or unexpected team member absences may delay assigned tasks, affecting dependent activities and the timely delivery of runnable

increments across the three iterations.

Probability - Medium: The four-member student team must complete multiple dependent development tasks within a seven-week schedule while managing other academic responsibilities.

Impact - High: Critical delays may postpone integration, testing, and runnable increments.

Strategy and action - Mitigation: Identify critical task dependencies, plan internal deadlines ahead of scheduled milestones, distribute work according to team availability, and maintain shared documentation to support task reassignment.

Monitoring and response: Review progress and availability weekly against Section 5.2.2; redistribute tasks and prioritize critical work when delays arise. **Owner:** Hila Kasa.

RSK-04 Underestimation of development effort

Description: Underestimated development, integration, or testing effort may delay dependent tasks and milestones.

Probability - Medium: The project includes multiple interdependent features and three difficulty levels, making accurate effort estimation challenging.

Impact - High: Accumulated delays may reduce available testing time and jeopardize the timely completion of required deliverables.

Strategy and action - Mitigation: Break complex work into manageable tasks, estimate effort, identify critical dependencies, and regularly track progress against the plan.

Monitoring and response: Review task completion and schedule deviations weekly against Section 5.2.2. Significant deviations trigger re-estimation, reprioritization of tasks, and workload redistribution while preserving mandatory requirements. **Owner:** Hila Kasa.

RSK-05 Incorrect Special Square Replacement

Description: Randomly placed Safe, Question, or Surprise squares may overlap fixed squares or violate difficulty-specific placement rules (US-02, US-08, US-12).

Probability: Medium - Placement must satisfy multiple position and quantity constraints.

Impact: High - Invalid board configurations violate mandatory requirements and may affect gameplay across difficulty levels.

Strategy and Action: Mitigation - Validate square counts, uniqueness, and allowed positions. Test multiple randomly generated boards for each difficulty level.

Monitoring and Response: Review board-generation test results in each relevant iteration. Invalid configurations trigger correction and repeated tests. **Owner:** Shira Marder.

RSK-06 Game History Data Loss or Corruption

Description: Completed-game results may be saved incorrectly, overwrite existing records, or violate the required CSV schema, affecting game history (US-11, US-14).

Probability: Medium - Persistence requires correct formatting, unique IDs, and file handling.

Impact: High - Lost or invalid records violate mandatory SRS requirements and prevent reliable history retrieval.

Strategy and Action: Mitigation - Validate the CSV schema, unique game IDs, and stored

values; test consecutive saves and history retrieval after restarting the application.
Monitoring and Response: Check stored records during Iterations 2-3. Missing or invalid records trigger correction and repeated persistence tests. **Owner:** Shira Marder.

RSK-07 Version Control and Merge Conflicts

Description: Simultaneous updates to shared MVC classes by multiple developers may result in complex Git merge conflicts, delaying the integration of the runnable increment.

Probability: Medium - Four developers working concurrently on interconnected game logic in a single repository increases collision chances.

Impact: High - Unresolved or improperly merged conflicts can break previously working code and severely delay the iteration deadline.

Strategy and action: Mitigation - Enforce strict feature branching, require pull request reviews before merging into the main branch, and communicate daily regarding which files are being modified.

Monitoring and response: The Scrum Master will review pull request status frequently. If integration blocks occur, stop new development until stabilization. **Owner:** Lana Admuz.

RSK-08 Cross-Platform UI Rendering Inconsistencies

Description: Java Swing interfaces may render incorrectly across computers due to screen resolutions or OS settings.

Probability: Medium - Java Swing relies on layout managers that can behave unexpectedly across different desktop environments if hard-coded dimensions are used.

Impact: Medium - The executable will still run, but visual defects violate the usability requirements and diminish the presentation quality.

Strategy and action: Mitigation - Rely on relative layout managers (e.g., BorderLayout, GridBagLayout) rather than absolute positioning, and test the UI on both Windows and Mac environments during Iteration 1.

Monitoring and response: Test UI rendering as part of the Definition of Done for any visual task. If overlapping occurs, standard frame sizes and layout managers will be refactored before the iteration milestone. **Owner:** Lana Admuz.

6.1 Process model – The Ludo project will follow Scrum across three iterations, each delivering a runnable increment. Each iteration includes planning, implementation, integration, peer reviews, and testing. The Product Owner will assess completed functionality against SRS requirements, acceptance criteria, and the Definition of Done.
Feedback and issues will inform subsequent iterations. The project will conclude with final system validation, documentation, and delivery. Work activities, milestones, and quality criteria are detailed in Sections 5.2.1, 5.2.2, and 5.3.4, respectively.

6.2 Methods, tools and techniques – Development will use Java, Eclipse, and Java Swing, with MVC separating game logic, interface, and control. Design patterns will support

modularity and reuse across difficulty levels. Git/GitHub feature branches and Pull Requests will support version control, peer reviews, and integration. History will follow the required CSV schema; question data will be stored separately, per the SRS. Testing will include functional, integration, regression, and system tests, with JUnit unit tests from Iteration 2. Runnable increments will be checked on multiple computers without an IDE. Development will follow the SRS and Section 5.3.4 quality criteria; supporting procedures appear in Sections 7.2-7.3.

7. Supporting process plan

7.2 Verification and Validation Plan – V&V covers the SRS, SDP, prototype, design, source code, test records, and runnable increments across all three iterations, using the tools in Section 6.2. **Verification:** Peer reviews use short checklists to assess document clarity, consistency, and course compliance, and design/code conformity with the SRS, MVC, and applicable design patterns. SRS identifiers link requirements, user stories, tasks, and tests. Reviews take place as relevant work products are completed. **Validation:** Prototype evaluations and increment demonstrations assess user tasks. Normal, edge-case, integration, and regression tests verify requirements, feature interactions, and existing functionality. Final validation covers all difficulty levels, 2-4 players, and execution on multiple compatible computers without an IDE. Unit testing begins in Iteration 2. **Responsibilities and records:** Developers self check; another team member reviews their work and independently executes relevant tests. The rotating Product Owner coordinates acceptance demonstrations. Shared records capture findings, severity, correction owners, expected/actual results, and follow-up outcomes. Corrections are checked before closure. Iteration 1 completion criteria and review intensity appear in Section 5.3.4

7.3 Documentation Plan – Deliverables include the SRS (IEEE 830), SDP (IEEE 1058), prototype documentation, and final software; internal records include JavaDoc, design notes, review findings, and tests. Assigned members prepare documents; another member reviews them before approval. Drafts are shared with the team before submission; approved versions are stored in shared Google Docs or GitHub. The SRS is complete; SDP and prototype are due November 1, 2026; later deliverables follow Section 5.2.2. All documentation follows applicable standards and course guidelines.

Team Work Division:

Hila Kasa - 6.2, 6.1, 5.2.1, 2 specific risks for 5.4
Shira Marder - 5.2.1, 5.2.2, 1.1.4, 2 specific risks for 5.4
Stav Elkriyef - 5.3.4, 7.2, 1.1.1 -1.1.3 , 2 specific risks for 5.4
Lana Admuz- 7.3, 5.3.1,2,3 , 2 specific risks for 5.4